Saved-post behavior when posts change

SOCIAL-P03 (#186)

Saved posts are stored as post IDs, so a user's saved list can point to a post that has since been deleted or hidden. This proposal covers how the save toggle (#149) and the Saved grid (#124) should handle that so they stay in sync after a retry, refresh or relaunch. It doesn't change the Firestore rules or the `SavedPost` model.

Recommendation

Build the Saved grid from the list of saved IDs (IDs-first), and only update that list once a save or unsave succeeds.

- Each saved ID gets exactly one entry in the grid. The toggle reads the same list, so the two always agree.
- If a saved post can't be found, its entry says “No longer available” and keeps an Unsave button, instead of showing a post card.
- Entries are sorted by post ID, which is the order `fetchSavedPostIds` already returns, so a refresh doesn't reorder the grid.

Comparison

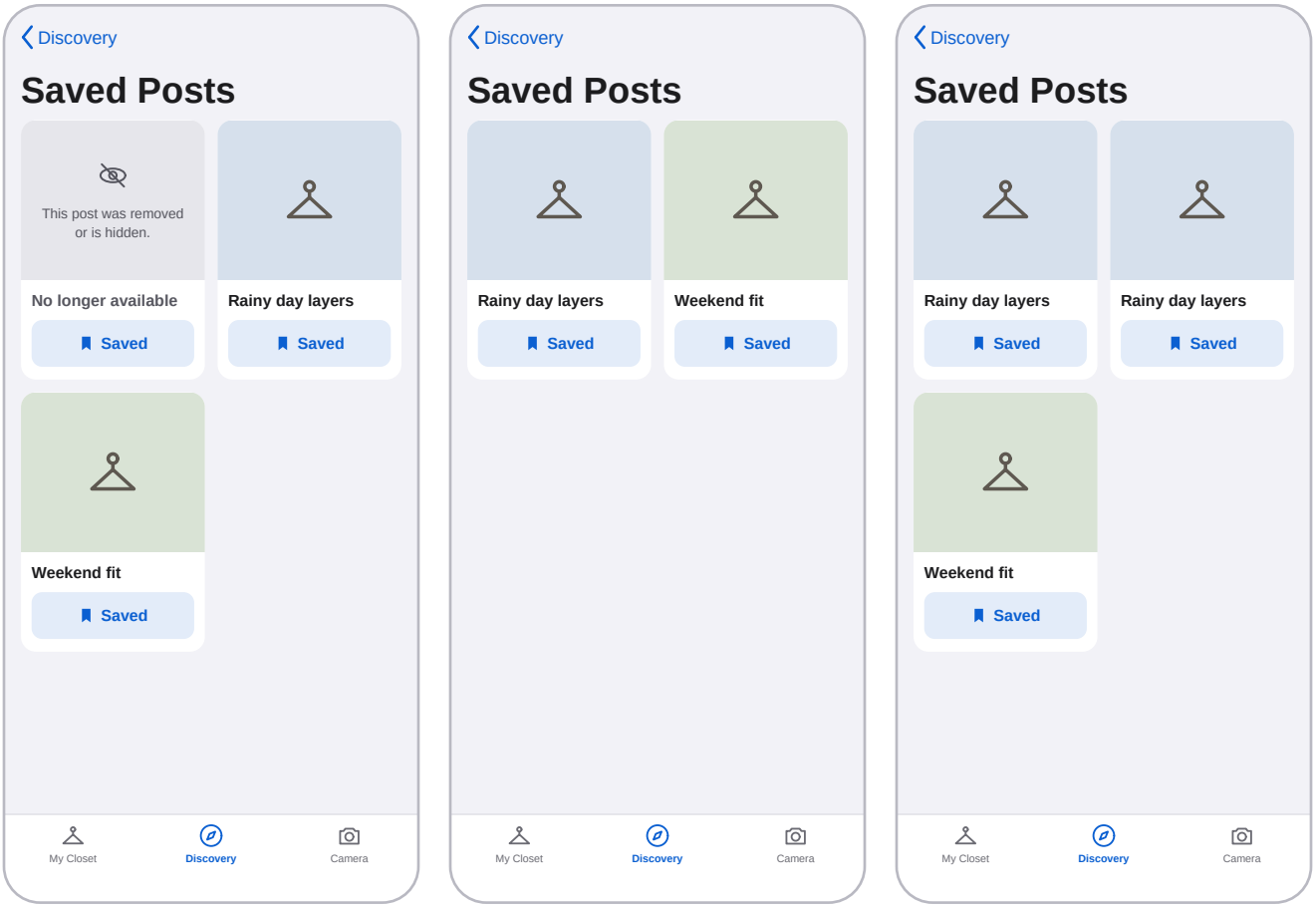
Both approaches use the same inputs: the saved IDs from `fetchSavedPostIds` (#122) and the posts the screen loads. The difference is how they're combined. Posts-first is how `filterSavedPosts` (#148) works today.

	Posts-first	**IDs-first**
How the grid is built	Keeps the loaded posts whose IDs are saved	One entry for each saved ID
A saved post is deleted	It drops out of the grid but stays saved, so it can't be unsaved from this screen	It shows as unavailable, with Unsave
The same post is loaded twice	Two cards	One entry
Order after a refresh	Can change	Stays the same
Prototype result	Fails 4 of 11 scenarios	Passes all 11

Mockups

All of these screens use the same three saved posts. post_001 was deleted by its author, while post_002 and post_003 still exist.

Figure 1. The Saved screen, proposed and current



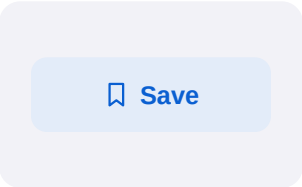
Proposed
post_001 shows as unavailable but can still be unsaved.

Current: deleted post
post_001 is gone from the grid even though it's still saved.

Current: duplicate post
post_002 was loaded twice, so it shows up twice.

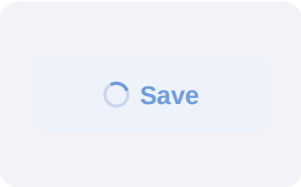
Figure 2. Save toggle (#149)

Not saved



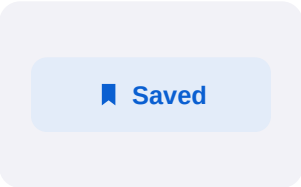
Tapping it saves the post.

Saving



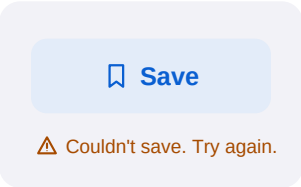
Disabled until the request finishes.

Saved



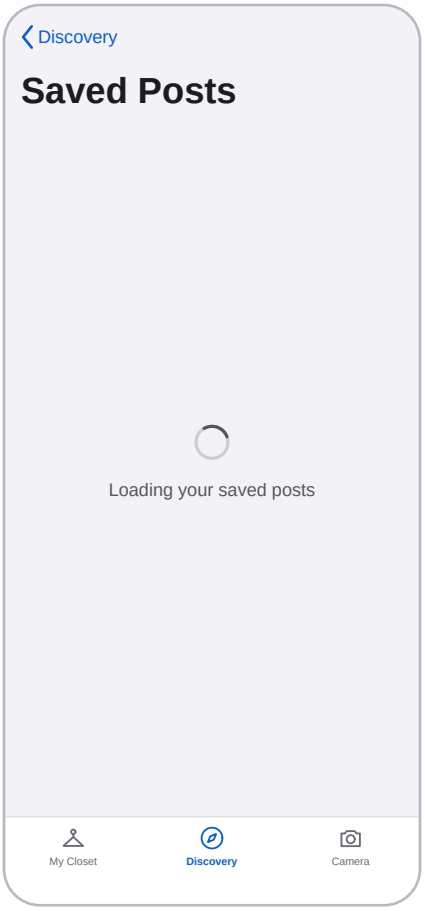
Shown once the save succeeds.

Failed

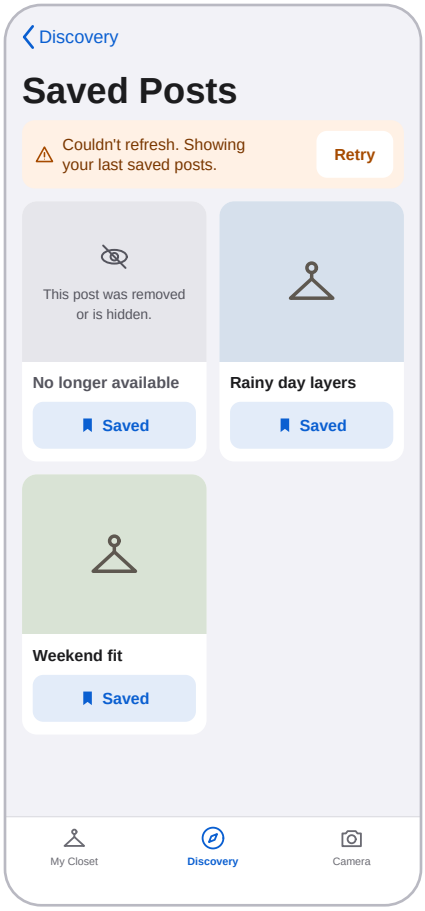


Goes back to how it was.

Figure 3. Loading and refresh errors



After a relaunch
Shows a loading state until saved posts are fetched, instead of “No saved posts yet.”



Refresh failed
Keeps the current grid and offers Retry.

State table

Event	Request	Toggle	Grid
Save	One `savePost` call	Save, then a spinner, then Saved	Adds the post once the save succeeds
Save again (already saved, or tapped while saving)	None. A retry writes to the same document, so there's never a duplicate	No change	No change
Unsave	One `removeSavedPost` call	Saved, then a spinner, then Save	Removes the entry once the unsave succeeds
Failed save or unsave	Only the failed call	Goes back to how it was, with an error	No change. If the request actually went through, the next refresh fixes it
Refresh	None	Matches the server	Matches the server. If the refresh fails, the grid stays as it is and offers Retry
Deleted post	None	Saved	Keeps the last card until the next refresh, then shows the post as unavailable
Relaunch	None	Loading until the first refresh	Loading until the first refresh, then matches the server
Sign-out	None	Cleared	Cleared

For the ticket's example: user_a saves post_001 twice, and the second save sends nothing because it's already saved. post_001 is then deleted. After a refresh the grid shows it as unavailable instead of a post card, and unsaving it sends one `removeSavedPost` call.

What the #94 rules need to enforce

For `users/{uid}/savedPosts/{postId}`:

1. Only the signed-in owner can read or write. Other users and signed-out clients are denied.
2. The document ID, `id` and `postId` match. `savePost` writes to `savedPost.id` and `removeSavedPost` deletes `postId`, so this keeps exactly one document per post.
3. `ownerId` equals `uid`, and the document only contains `id`, `schemaVersion` (1), `ownerId`, `postId` and `createdAt` (a timestamp).
4. Saving a post that's already saved succeeds. `savePost` sends a new `createdAt` each time, so either let `createdAt` change on update or have `savePost` keep the original date.
5. Deleting a document that doesn't exist succeeds, so unsaving twice is harmless.
6. The emulator tests cover the owner, another user and a signed-out client.

Optionally, a save could also require `posts/{postId}` to exist. Right now the rules deny all access to `savedPosts`, so saving and unsaving will fail until these rules are added.

What the Saved screen needs after a relaunch

Nothing is kept in memory, so the screen reloads everything from the server:

1. The signed-in user's ID. Saved-post state is reset on sign-out.
2. The saved IDs from `fetchSavedPostIds`. If that fails, the screen shows an error with Retry, not “No saved posts yet.”
3. The posts for those IDs, looked up by ID. `fetchRecentPosts` only returns the 20 newest posts, so older saved posts would show as unavailable. A function for this doesn't exist yet.
4. A loading state until the first fetch finishes.
5. The post itself when saving from a post card, so the grid can show it right away.
6. `SavedPost` records with `id` and `postId` set to the post's ID and `ownerId` set to the signed-in user.

Other changes

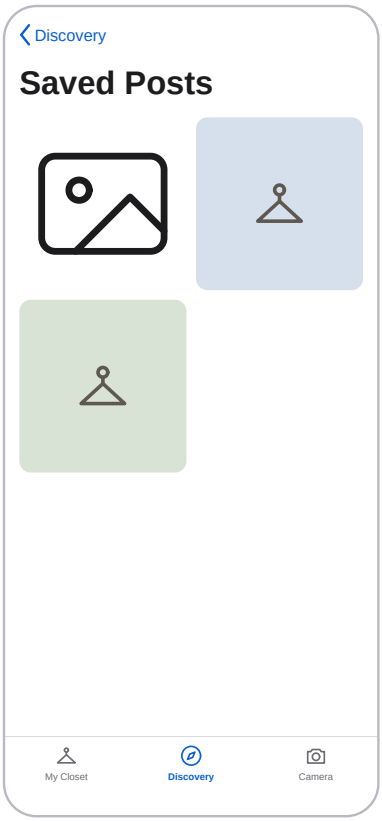


Figure 4. Current #124 grid
The deleted post shows up as a large photo placeholder.

`SavedPostGrid` (#124) shows a placeholder photo for any saved post without an image, so a deleted post looks like a normal card that can be tapped. It needs a separate unavailable entry.

The Saved grid shouldn't use `filterSavedPosts` (#148), since it drops posts that are missing and repeats posts that are loaded twice.

Prototype

The `prototype` folder contains a small Swift version of the IDs-first state (`SavedPostsState`) and a check that runs both approaches through 11 scenarios using in-memory test data. It sits next to this document in `docs/proposals/saved-post-behavior` and isn't part of the app. To run it, from the repository root:

```
xcrun swiftc dressed/Common/Models/*.swift dressed/Discovery/SavedPostFilters.swift \
  docs/proposals/saved-post-behavior/prototype/*.swift -o /private/tmp/saved-post-check
/private/tmp/saved-post-check
```

IDs-first passes all 11 scenarios. Posts-first fails four of them: order after refresh, deleted post, the ticket example, and the same post loaded twice. `SavedPostsState` also compiles with the app's Swift settings, so it can move into `dressed/Discovery/ViewModels` if this is approved.